Davante

11

PROGRAMACIÓN

Mecanismos de abstracción basados en herencia

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Clases abstractas 5

/ 3. Métodos abstractos 5

/ 4. Caso práctico 1: “Instrumentos” 6

/ 5. Comprobación de tipos. La sentencia instanceof 7

/ 6. Casting entre objetos 8

/ 7. Definición e implementación de interfaces 9

/ 8. Atributos y métodos de una interfaz 10

8.1. Atributos de una interfaz 10

8.2. Métodos de una interfaz 11

/ 9. Herencia y variables de tipo interfaz 11

/ 10. Clases anónimas 13

/ 11. Clase abstracta vs. interfaz 14

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 12. Caso práctico 2: “Piano” 15

/ 13. Interfaces de la API 16

13.1. Comparable 16

13.2. Comparator 17

/ 14. Resumen y resolución del caso práctico de la unidad 18

/ 15. Bibliografía 19

OBJETIVOS

Conocer y utilizar las clases abstractas.

Emplear las clases abstractas en la selección dinámica de métodos.

Conocer la idea y la necesidad de las interfaces y distinguirlas de las clases abstractas.

Diseñar interfaces para operaciones específicas.

Emplear interfaces como tipo para conseguir la selección dinámica de métodos.

Conocer algunas interfaces importantes de la API.

/ 1. Introducción y contextualización práctica

Las clases abstractas y las interfaces desempeñan un papel crucial en la programación orientada a objetos, ya que proporcionan mecanismos para la abstracción, la reutilización y la estandarización del código.

Permiten definir métodos sin implementación concreta, lo que fomenta la creación de jerarquías de clases coherentes y la especificación de comportamientos comunes.

Por otro lado, las interfaces establecen contratos que las clases deben cumplir, lo que promueve la interoperabilidad y la cohesión del código.

En conjunto, las clases abstractas y las interfaces son fundamentales para lograr el polimorfismo, permitiendo que objetos de diferentes clases puedan ser tratados de manera uniforme.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

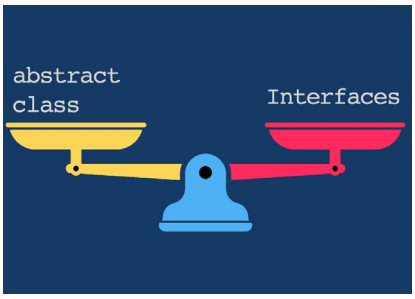


Fig.1. Clases abstractas e interfaces.

Audio intro. "Herencia múltiple e interfaces"

https://on.soundcloud.com/31zyM7gr2BUkK7tE6

/ 2. Clases abstractas

Cuando ya tengamos cierto manejo de la utilización de la herencia, llegaremos a un punto en el que empezaremos a definir clases, que van a contener funcionalidades comunes de otras, de las que queremos que hereden, pero no tendrá sentido instanciar un objeto de ellas.

Por ejemplo, podemos crear las clases Mamífero y Reptil, ambas con una serie de propiedades comunes a todos los mamíferos y reptiles, para luego crear las clases Perro, Gato y Serpiente, que heredarán de las anteriores.

Ahora nos preguntamos, ¿declararemos objetos de las clases Mamífero y Reptil, o solo de las clases Perro, Gato y Serpiente? La respuesta está bastante clara, y es que solo vamos a necesitar objetos de Perro, Gato y Serpiente, ya que serán animales concretos, mientras que crear un mamífero o un reptil puede ser confuso, ya que no sabemos a qué tipo de animal se van a referir.

En este ejemplo, las clases Mamífero y Reptil van a ser clases abstractas, clases de las que no vamos a instanciar nunca un objeto, lo cual no tendría sentido.

Cuando declaramos una clase abstracta, estamos indicando que de la misma no se van a poder instanciar objetos, pero sí que se va a poder heredar de ella. Esto nos servirá para agrupar una serie de funcionalidades comunes.

Declaramos una clase abstracta en **Java** mediante la palabra reservada **'abstract'**, d**elante del nombre de la clase** en cuestión. A partir de ese momento, no se podrán instanciar objetos de dicha clase, aunque sí se podrán implementar los constructores en una clase abstracta.

Por tanto, **una clase abstracta no puede ser instanciada directamente con la palabra clave 'new'**. Debe ser subclasificada (heredada) y, al igual que una clase normal, una clase abstracta puede contener variables de instancia, métodos de instancia, métodos estáticos, constructores y heredar de otra clase sin inconveniente alguno.

```
abstract class Mamifero {
    private String nombre;
    private int edad;

    public Mamifero(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
```

Fig.2. Clase abstracta Mamífero.

/ 3. Métodos abstractos

En Java, **un método abstracto es un método declarado sin una implementación en la clase en la que se define**. Estos métodos están destinados a ser implementados por las subclases que heredan de la clase que los declara.

Para declarar un método como abstracto, se utiliza la palabra clave **'abstract'**. Los métodos abstractos cumplen las siguientes **restricciones**:

- **No tienen cuerpo**: No poseen una implementación definida dentro de la clase abstracta que los declara. Se declaran solo con la firma del método y terminan con un punto y coma.
- **Se definen en clases abstractas**: Se declaran generalmente en clases abstractas. Sin embargo, también pueden existir en interfaces, donde todos los métodos son abstractos por defecto.
- **Deben ser implementados por las subclases**: Las subclases que heredan de una clase abstracta que contiene métodos abstractos deben proporcionar una implementación concreta para esos métodos. Si una subclase no implementa todos los métodos abstractos heredados, la subclase también debe ser declarada abstracta.

Esto nos va a servir para que, cuando tengamos clases que hereden de la clase abstracta y tengan que implementar los métodos abstractos, cada una lo haga de una forma diferente, es decir, vamos a tener el **mismo método base, pero con diferentes implementaciones**.

Una clase abstracta puede no contener ningún método abstracto, solamente métodos 'normales', pero en el caso de necesitar un método abstracto en una clase, eso hará que esta ya tenga que ser abstracta forzosamente, como se muestra en la imagen.

```
abstract class Forma {
    // Método abstracto para calcular el área
    public abstract double calcularArea();
}

class Circulo extends Forma {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    // Implementación del método abstracto calcularArea() para el círculo
    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

Fig.3. Métodos abstractos.

En este ejemplo, la clase Forma es una clase abstracta que contiene un método abstracto calcularArea(). La subclase Circulo hereda de Forma y proporciona una implementación concreta para el método calcularArea(). La subclase Forma no puede ser instanciada directamente, pero sus subclases pueden ser utilizadas polimórficamente a través de una referencia de tipo Forma.

Audio 1. "Heredando de una clase abstracta"
https://bit.ly/3brHz1v

/ 4. Caso práctico 1: "Instrumentos"

Planteamiento: Las clases abstractas son fundamentales, ya que permiten definir una interfaz común para un grupo de clases relacionadas, asegurando que todas las subclases implementen ciertos métodos específicos (métodos abstractos). Esto fomenta la reutilización del código. Además, las clases abstractas facilitan el diseño de arquitecturas más robustas y flexibles, permitiendo a los desarrolladores trabajar con clases derivadas de manera uniforme, sin preocuparse por los detalles específicos de implementación.

Nudo: Crea una clase abstracta Instrumento que almacene las notas(enumerados) musicales de una melodía a través del método add(Nota n). Este método se encargará de añadir las notas a la melodía, mientras que el método Interpretar será el encargado de mostrar las notas según el instrumento que haga su lectura, es decir, será un método abstracto.

Desenlace: La siguiente imagen muestra una posible solución al algoritmo que se plantea:

```
public enum Nota{DO, RE, MI, FA, SOL, LA, SI}

public abstract class Instrumento {

    protected Nota [] melodia;

    public Instrumento (){
        melodia = new Nota[0];
    }

    public add(Nota n){
        melodia = Arrays.copyOf(melodia, melodia.length + 1);
        melodia[melodia.length - 1] = n;
    }

    abstract void interpretar();
}
```

Fig.4. Clase abstracta Instrumento.

/ 5. Comprobación de tipos. La sentencia instanceof

Cuando utilicemos **polimorfismo**, vamos a tener **varias clases** que **heredan de una misma clase padre, pero con diferente funcionalidad en sus métodos**.

Pongamos un ejemplo. Imaginemos que tenemos la clase Animal, la cual es abstracta y tiene un método desplazarse, abstracto también. Declaramos también las clases Perro, Gato y Serpiente, todas heredando de la clase Animal, e implementamos el método desplazarse indicando que un perro anda, un gato camina y una serpiente repta.

Imaginemos que declaramos varios objetos de las clases Perro, Gato y Serpiente, cada uno con sus propios datos.

Ahora vamos a aplicar una de las mayores ventajas del polimorfismo. Si bien en la unidad dedicada a los *arrays* vimos que en un array solo podíamos tener datos del mismo tipo, ¿cómo podemos aprovechar el polimorfismo? Pues bien, si declaramos un array de objetos de tipo Perro, solo podremos almacenar objetos de ese tipo, pero ¿qué ocurre si creamos un array de objetos de tipo Animal?

El hecho de que la clase Animal sea abstracta no quiere decir que no se puedan crear arrays que almacenen ese tipo, es más, en un array de datos de tipo Animal, podremos guardar objetos de cualquiera de las clases que hereden de ella, es decir, podremos guardar, en un array de tipo Animal, objetos de tipo Perro, Gato y Serpiente.

Sin embargo, ahora tenemos un problema. Cuando obtengamos un dato de ese array, será un objeto de tipo Animal, ¿y si necesitamos llamar a algún método específico de las clases derivadas? Esto lo podremos solucionar con la instrucción instanceof, mediante la cual podremos saber de qué tipo es un objeto.

Su uso es el siguiente:

```
if( nombreobjetoinstanceofNombreClase )
```

¡! Sabías que...

Existe una mejor alternativa que es definir una interfaz que agrupe este comportamiento en lugar de comprobar cada tipo. Luego en lugar de hacer comprobaciones con instanceof, podemos recorrer el array y llamar al método de la interfaz si el objeto la implementa.

En la siguiente imagen se puede ver un ejemplo de su uso. En el caso de que el objeto sea del tipo de la clase que indica la expresión, tomará el valor de verdadero, mientras que si no, valdrá falso.

```
public static void main(String[] args) {
    FiguraGeometrica figura = new Cuadrado(4);

    if (figura instanceof Cuadrado) {
        System.out.println("La figura instanciada es un cuadrado.");
    } else {
        System.out.println("La figura instanciada NO es un cuadrado.");
    }
}
```

Fig.5. Método instanceof.



Vídeo 1. "Ejemplo de uso de instanceof"
https://bit.ly/3br2ba5



/ 6. Casting entre objetos

Como ya vimos en unidades pasadas, los *castings* son conversiones de tipo entre variables, en las que podría producirse algún tipo de pérdida de información. También vimos que se podían realizar entre variables de tipo primitivo, siempre y cuando estas fuesen compatibles.

Vamos a ver, a continuación, que, **entre objetos, también se van a poder realizar castings**, y, para ello, necesitamos **utilizar la herencia y el polimorfismo**.

Al igual que en las variables de tipos primitivos, los castings entre objetos se van a poder realizar cuando las clases sean compatibles, o, lo que es lo mismo, cuando estemos trabajando con una jerarquía de clases.

El mayor partido lo vamos a sacar cuando queramos objetos de un tipo en concreto, ya que, si hacemos el casting a la clase padre, no vamos a poder utilizar los métodos de las clases hijas.

Imaginemos que tenemos la misma jerarquía de clases que en el punto 5, es decir, tenemos una clase Animal y tres clases hijas, Perro, Gato y Serpiente.

Volvemos al caso de crear un array de objetos de tipo Animal e introducimos varios objetos de tipo Perro, Gato y Serpiente.

Al recorrer el array (siguiente imagen), mediante la instrucción instanceof, podremos saber cuándo tenemos un objeto de cada uno de los tipos, pero ¿qué ocurre si queremos invocar a un método de una de las clases hijas? Pues como tenemos objetos compatibles entre sí, podremos hacer un casting a la clase que nos interese.

Al igual que con las variables primitivas, pondremos entre paréntesis el nombre de la clase a la que queremos hacer el casting.

```
for(Animal ani : animales)
{
    if( ani instanceof Perro )
    {
        Perro p = (Perro)ani;
        p.metodoPerro();
    }
    if( ani instanceof Gato )
    {
        Gato p = (Gato)ani;
        p.metodoGato();
    }
    if( ani instanceof Serpiente )
    {
        Serpiente p = (Serpiente)ani;
        p.metodoSerpiente();
    }
}
```

Fig.6. Ejemplo de casting entre objetos.

/ 7. Definición e implementación de interfaces

Una interfaz es una serie de acciones relacionadas entre sí, que va a poder ejecutar un objeto.

Concretamente, **una interfaz es una clase que únicamente puede tener variables constantes y métodos sin implementación**, pero que no son abstractos.

Básicamente, en una interfaz únicamente existen los prototipos de las funcionalidades, es decir, de los métodos, que más tarde se agregarán a otra clase.

Las interfaces proporcionan las siguientes **ventajas**:

- Organización y estructuración de código.
- Las constantes que se creen en una interfaz estarán disponibles para todas aquellas clases que la usen.
- Se obliga a que las clases que la utilizan empleen el mismo método.
- Permiten establecer una relación entre clases que aparentemente no están relacionadas.

Para declarar una interfaz en Java, utilizamos la palabra reservada:

```
interface: [public/private] interface NombreInterfaz
```

El cuerpo de la interfaz irá entre llaves ({ }) y en él se declararán las variables constantes y los métodos que se necesite.

Para poder usar una interfaz, tendremos que 'implementarla', esto se hace mediante la palabra reservada:

```
implements [public/private] classNombreClaseimplementsNombreInterfaz
```

Al contrario que con la herencia, que únicamente se puede heredar de una clase, con las interfaces podremos implementar todas las que necesitemos, aunque habrá que implementar los métodos de todas ellas. Para ello separaremos las interfaces a implementar mediante comas, como se muestra en la imagen.

¡! Sabías que...

A partir de Java 8, las interfaces pueden contener implementaciones por defecto, lo que ha difuminado la diferencia en el uso entre clases abstractas e interfaces.

```
[public/private] classNombreClaseimplements NombreInterfaz1,
NombreInterfaz2, NombreInterfaz3
```

A continuación podrás ver una imagen sobre el punto anterior:

```
public interface IReproductor {
    public void play();

    public void pause();

    public void stop();
}

class VideoPlayer implements IReproductor {
    @Override
    public void play() {...3 lines }

    @Override
    public void pause() {...3 lines }

    @Override
    public void stop() {...2 lines }
}

class AudioPlayer implements IReproductor {
    @Override
    public void play() {...2 lines }
```

Fig.7. Ejemplo de interfaz.

/ 8. Atributos y métodos de una interfaz

Como ya hemos dicho previamente, las interfaces pueden tener variables constantes y métodos sin implementación.

8.1. Atributos de una interfaz

Los **atributos no se pueden definir directamente en una interfaz**, ya que están asociados con el estado interno de un objeto, y las interfaces en Java están diseñadas principalmente para especificar comportamiento, no estado. Sin embargo, se puede definir constantes en una interfaz, las cuales actúan como variables finales estáticas.

Observemos el siguiente código:

```
interface Debrapp {
        doubleversion = 2.3;
        voidjugar();
}
```

En el código, la variable versión, es una constante que puede ser accedida desde cualquier clase que implemente esta interfaz, mientras que jugar() es un método abstracto, es decir, no tiene una implementación en la interfaz misma.

8.2. Métodos de una interfaz

La siguiente imagen muestra los tres tipos de métodos que pueden contener las interfaces, además de los métodos abstractos tradicionales:

```java
public interface MiInterfaz {
    // Método por defecto
    default void metodoPorDefecto() {
        System.out.println("Método por defecto de la interfaz");
        metodoPrivado();
    }

    // Método estático
    static void metodoEstatico() {
        System.out.println("Método estático de la interfaz");
    }

    // Método privado
    private void metodoPrivado() {
        System.out.println("Método privado de la interfaz");
    }
}
```

Fig.8. Métodos de una interfaz.

- **Métodos por defecto** (*default methods*): Tienen una implementación por defecto dentro de la interfaz. Permiten añadir nuevos métodos a interfaces existentes sin romper la compatibilidad con las implementaciones de las interfaces. Se declaran usando la palabra clave 'default'.

- **Métodos estáticos** (*static methods*): Están asociados a la interfaz en sí misma, no a las instancias de las clases que implementan la interfaz. Se pueden llamar utilizando el nombre de la interfaz seguido del nombre del método. Se declaran usando la palabra clave 'static'.

- **Métodos privados** (*private methods*): Son útiles para encapsular la lógica común dentro de la interfaz, que solo se utilizará dentro de otros métodos de la misma interfaz. No se pueden llamar directamente desde fuera de la interfaz ni desde las clases que la implementan. Se declaran usando la palabra clave 'private'.

/ 9. Herencia y variables de tipo interfaz

Cuando hablamos de **variables de tipo interfaz en Java,** nos referimos a la **capacidad de declarar una variable cuyo tipo es una interfaz en lugar de una clase concreta**. Esto permite la flexibilidad en la asignación de objetos de diferentes clases que implementan esa interfaz a esa variable.

La **herencia** en este contexto se refiere a la **capacidad de una interfaz de extender otra interfaz**, lo cual es similar a la herencia de clases, pero en el contexto de interfaces.

Vamos a crear dos interfaces y una tercera que hereda de las dos primeras y, a continuación, vamos a crear una clase Circulo que implementa la tercera interfaz: FiguraColoreada (imagen):

```
interface Figura {
    double area();
}

interface Color {
    void setColor(String color);
}

interface FiguraColoreada extends Figura, Color {
    // Esta interfaz extiende Figura y Color
}
```

```
class Circulo implements FiguraColoreada {
    private double radio;
    private String color;

    public Circulo(double radio) {
        this.radio = radio;
    }

    @Override
    public double area() {
        return Math.PI * radio * radio;
    }

    @Override
    public void setColor(String color) {
        this.color = color;
    }
}
```

Fig.9. Herencia de interfaces y clase que las implementa.

Una vez creada la clase Circulo que implementa la interfaz FiguraColoreada, podemos crear variables de tipo interfaz para referenciar objetos de la clase Circulo, como se muestra en el siguiente fragmento de código (imagen):

```
public class Main {
    public static void main(String[] args) {
        FiguraColoreada miFigura = new Circulo(5.0);
        miFigura.setColor("Rojo");
        System.out.println("Área de la figura: " + miFigura.area());
    }
}
```

Fig.10. Uso de interfaces Circulo.

En este ejemplo podemos observar lo siguiente:

- Se declara una variable miFigura de tipo FiguraColoreada, que es una interfaz que hereda tanto Figura como Color.
- Se crea una instancia de Circulo y se le asigna a miFigura, ya que Circulo implementa FiguraColoreada.
- Esto permite que se pueda llamar a métodos tanto de Figura como de Color a través de miFigura, ya que tiene esos métodos debido a la herencia de las interfaces.

> **¡! Sabías que...**
>
> Al no existir herencia múltiple en Java, una clase puede heredar de una sola clase abstracta, pero puede implementar múltiples interfaces simulando la herencia múltiple.

/ 10. Clases anónimas

De manera general, las interfaces son implementadas por clases que hacen uso de ellas y de las que se crearán distintos objetos. Sin embargo, se puede dar el caso de querer implementar una interfaz en un único lugar, sin necesidad de crear una clase para un solo objeto del que interesan únicamente las funcionalidades de la interfaz.

En estos casos, Java nos permite crear en el fragmento de código donde se va a implementar un objeto de una clase sin nombre, es decir, anónima. **Las clases anónimas son útiles para crear implementaciones de interfaces de manera concisa y en el lugar donde se necesitan**. Cuando se utilizan clases anónimas para implementar una interfaz, se puede acceder a los miembros de la misma dentro de la clase anónima, de igual manera que se haría en una clase normal.

A continuación se va a realizar un ejemplo que implementa el acceso a los miembros de la interfaz desde la clase anónima. Partimos de la interfaz Calculadora con un único método calcular, que toma dos números y devuelve un resultado:

```
interface Calculadora {
    double calcular(double a, double b);
}
```

Fig.11. Interfaz Calculadora.

El siguiente paso es emplear una clase anónima para implementar esta interfaz y realizar una suma como se muestra en la imagen:

```
public class Main {
    public static void main(String[] args) {
        // Implementación de la interfaz utilizando una clase anónima
        Calculadora suma = new Calculadora() {
            @Override
            public double calcular(double a, double b) {
                return a + b;
            }
        };

        // Uso de la implementación para realizar una suma
        double resultado = suma.calcular(5, 3);
        System.out.println("El resultado de la suma es: " + resultado);
    }
}
```

Fig.12. Creación de clase anónima.

Se puede observar que:

- Se ha creado una instancia de Calculadora empleando una clase anónima para implementar la interfaz.
- Dentro de la clase anónima, se proporciona una implementación del método calcular, que en este caso realiza una suma de los dos números proporcionados.
- Se puede acceder al método calcular de la interfaz Calculadora desde dentro de la clase anónima y utilizarlo como cualquier otro método.

/ 11. Clase abstracta vs. interfaz

Cuando hablamos de interfaces, se pueden confundir con las clases abstractas, ya que, vista la definición de una interfaz, estas y las clases abstractas **parecen iguales**. Este error es muy común cuando se está aprendiendo el lenguaje, ya que prácticamente parecen lo mismo, salvo por **leves diferencias**.

1. **Métodos abstractos**:
   - » **Clase abstracta**: Puede contener métodos abstractos (sin cuerpo) y métodos concretos (con cuerpo).
   - » **Interfaz**: Solo puede contener métodos abstractos. A partir de Java 8, también puede incluir métodos concretos si están marcados con la palabra clave 'default'.
2. **Herencia**:
   - » **Clase abstracta**: Puede extender solo una clase (abstracta o concreta), pero puede implementar múltiples interfaces.
   - » **Interfaz**: Puede extender múltiples interfaces, pero no una clase. No puede implementar otra interfaz.
3. **Constructores**:
   - » **Clase abstracta**: Puede tener constructores, que pueden ser invocados por las subclases.
   - » **Interfaz**: No puede tener constructores.
4. **Estado**:
   - » **Clase abstracta**: Puede tener campos (variables de instancia), que pueden ser heredados por las subclases.
   - » **Interfaz**: No puede tener campos. Solo puede contener constantes (variables estáticas final) y métodos.
5. **Uso**:
   - » **Clase abstracta**: Se utiliza cuando se quiere definir una clase base que proporciona cierta implementación predeterminada y que puede ser extendida por subclases.
   - » **Interfaz**: Se emplea cuando se quiere definir un conjunto de métodos que deben ser implementados por diferentes clases, independientemente de su jerarquía de herencia.

En resumen, **las clases abstractas se utilizan para definir una estructura común y proporcionar implementaciones predeterminadas, mientras que las interfaces se usan para definir un contrato común entre las clases.** Ambas son herramientas importantes en la programación orientada a objetos y tienen su propio conjunto de casos de uso y ventajas.

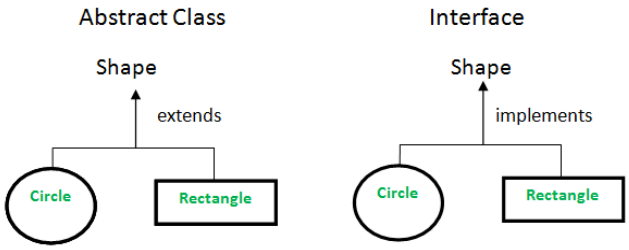


Fig.13. Clase abstracta vs. interfaz.

Recuerda...

La principal diferencia entre clases abstractas e interfaces es que las primeras definen qué es una clase, mientras que las segundas definen cómo se comporta una clase.

/ 12. Caso práctico 2: "Piano"

Planteamiento: Los métodos abstractos son esenciales, porque definen un contrato que las clases deben cumplir, asegurando que implementen comportamientos específicos. Al declarar métodos sin cuerpo en clases abstractas, se establece una estructura común que promueve la interoperabilidad entre distintas clases. Esto no solo mejora la organización y el mantenimiento del código, sino que también facilita la extensión y escalabilidad del sistema, permitiendo añadir nuevas funcionalidades con mínima modificación del código existente.

Nudo: Crea una clase Piano que herede de la clase abstracta Instrumento. A continuación, crea una melodía e interprétala mediante la clase Piano.

Desenlace: La siguiente imagen muestra una posible solución al algoritmo que se plantea:

```java
public class Piano extends Instrumento {

    public Piano() {
        super();
    }

    @Override
    public void interpretar() {
        for (Nota nota : melodia) {
            switch (nota) {
                case DO:
                    System.out.print("do ");
                    break;
                case RE:
                    System.out.print("re ");
                    break;
                case MI:
                    System.out.print("mi ");
                    break;
                case FA:
                    System.out.print("fa ");
                    break;
                case SOL:
                    System.out.print("sol ");
                    break;
                case LA:
                    System.out.print("la ");
                    break;
                case SI:
                    System.out.print("si ");
                    break;
            }
        }
        System.out.println("");
    }
}
```

```java
public static void main(String[] args) {
    Nota cancion[] = {Nota.DO, Nota.FA, Nota.RE, Nota.FA, Nota.LA};
    Piano p = new Piano();
    for (Nota nota : cancion) {
        p.add(nota);
    }
    p.interpretar();
}
```

Fig.14. Clase Piano y clase principal.

/ 13. Interfaces de la API

En Java existen multitud de interfaces que proporcionan funcionalidades esenciales para el desarrollo de aplicaciones, entre las que destacan las siguientes para la ordenación.

13.1. Comparable

La interfaz Comparable en Java se utiliza para definir un orden natural de los objetos de una clase. Esta interfaz tiene un solo método llamado compareTo, que toma un argumento de tipo Object y devuelve un entero que indica si el objeto actual es menor, igual o mayor que el objeto pasado como argumento.

La firma del método compareTo es la siguiente:

```
publicintcompareTo(T objeto)
```

Donde T es el tipo de objeto con el que se está comparando.

La implementación de este método debe seguir la convención de retorno de valores:

- Un valor negativo si el objeto actual es menor que el objeto pasado como argumento.
- Cero si el objeto actual es igual al objeto pasado como argumento.
- Un valor positivo si el objeto actual es mayor que el objeto pasado como argumento.

Por ejemplo, supongamos que tenemos una clase Persona y queremos que las instancias de la misma se puedan comparar por su edad. Se podría implementar la interfaz Comparable de la siguiente manera, como se muestra en la figura:

```java
public class Persona implements Comparable<Persona> {
    private String nombre;
    private int edad;

    // Constructor, getters y setters

    @Override
    public int compareTo(Persona otraPersona) {
        return Integer.compare(this.edad, otraPersona.edad);
    }
}
```

Fig.15. Implementación Comparable.

En este ejemplo, la implementación del método compareTo compara las edades de dos personas. Utilizaremos el método Integer.compare para comparar las edades y devolver el resultado.

Luego, podemos usar métodos como Collections.sort() para ordenar una lista de objetos Persona utilizando el orden natural definido por el método compareTo.

```
Collections.sort(personas);
```

Donde personas es una lista de objetos persona.

```
List<Persona> personas = new ArrayList<>();
```

13.2. Comparator

La interfaz Comparator en Java se utiliza para definir múltiples formas de ordenar objetos. A diferencia de la interfaz Comparable, que se utiliza para definir un único orden natural para los objetos de una clase, Comparator permite definir múltiples criterios de ordenación o un orden personalizado para una clase, sin modificar la implementación de la propia clase.

La interfaz Comparator tiene un solo método llamado 'compare', que toma dos argumentos de tipo T (el tipo de objeto que se va a comparar) y devuelve un entero que indica si el primer objeto es menor, igual o mayor que el segundo objeto.

La firma del método compare es la siguiente:

```
intcompare(T obj1, T obj2)
```

Donde T es el tipo de objeto que se está comparando.

Al igual que con la interfaz Comparable, la implementación del método compare debe seguir la convención de retorno de valores:

- Un valor negativo si el primer objeto es menor que el segundo.
- Cero si los dos objetos son iguales.
- Un valor positivo si el primer objeto es mayor que el segundo.

A continuación se va a realizar un ejemplo(imagen) de cómo usar la interfaz Comparator para ordenar objetos de la clase Persona por su edad de forma descendente:

```java
import java.util.Comparator;

public class Persona {
    private String nombre;
    private int edad;

    // Constructor, getters y setters

    // Método estático que devuelve un Comparator para ordenar por edad de forma descendente
    public static Comparator<Persona> edadComparatorDescendente = new Comparator<Persona>() {
        @Override
        public int compare(Persona persona1, Persona persona2) {
            return Integer.compare(persona2.edad, persona1.edad); // Orden descendente
        }
    };
}
```

Fig.16. Implementación de Comparator.

En este ejemplo, se define un Comparator llamado edadComparatorDescendente que compara dos objetos Persona por su edad en orden descendente. Luego, se puede usar este Comparator para ordenar una lista de personas de la siguiente manera:

```
Collections.sort(personas, Persona.edadComparatorDescendente);
```

Donde personas es una lista de objetos persona.

```
List<Persona> personas = new ArrayList<>();
```

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema se ha comenzado introduciendo el concepto de **clase abstracta**. Estas clases proporcionan una base para la creación de **clases hijas** que pueden heredar atributos y métodos permitiendo definir **métodos abstractos**, lo que obliga a las clases hijas a implementarlos.

El siguiente punto se ha dedicado a las **interfaces**. Se trata de contratos que especifican métodos que deben ser implementados por las clases que las utilizan, lo que las distingue de las clases abstractas. Estos elementos son útiles para diseñar operaciones específicas y se emplean como tipo para lograr la selección dinámica de métodos.

Al emplearse ambos mecanismos como tipos para lograr el polimorfismo, se ha incluido un apartado para diferenciar un tipo de otro mediante el uso de **instanceof** y un **casting entre objetos**.

Por último, se han introducido dos de las interfaces más importantes de la API de Java, como son **Comparable** y **Comparator**.

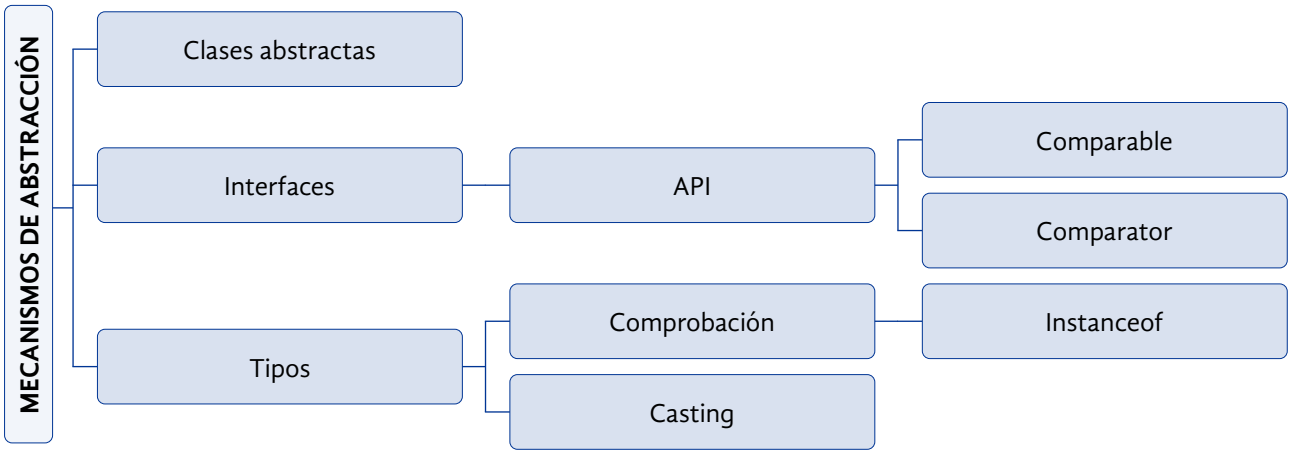


Fig.17. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Este es uno de problemas más comunes que vamos a encontrar al trabajar con herencia: Existen varias clases que ya han heredado, pero se necesita que estas tengan una funcionalidad común, y como ya han heredado y en Java no se permite la herencia múltiple, no se podrá hacer una clase en la que esté dicha funcionalidad, declararla como abstracta y hacer que hereden de ella.

La solución a este problema son las interfaces. Como ya se ha comentado, en una interfaz se pueden declarar todas las funciones necesarias, además estarán sin implementar, y más tarde hacer que una clase implemente dicha interfaz, teniendo que implementar forzosamente dichos métodos. Este comportamiento nos va a salvar el hecho de que Java no pueda implementar una herencia múltiple.

/ 15. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (*Vol. 1: Fundamental Algorithms*) (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.